Linked List

Quality in-depth

≃ array
≃ Linear data structure

Before going forward to understand linked list, let us see problem with array

Prove how python list is dynamic array.
- sys.getsizeof( )

loop to show the same.

Explain wrt array

Explain wrt list i.e. dynamic array.

Main problem, cannot add more elements than specified.



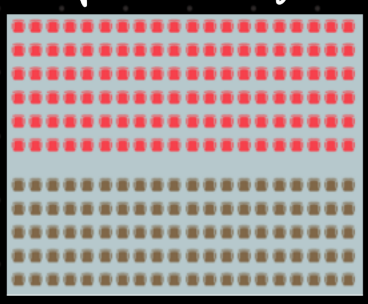
This space might not be free thus cont.

Problem is because array elements are continously placed.

so
- insertion / deletion
- memory wasted a lot

Linked list so now created to tackle this problem

⇒ add element as per requirement



Random allocation of space (non-continous)

Explain using student on a trip. / movie



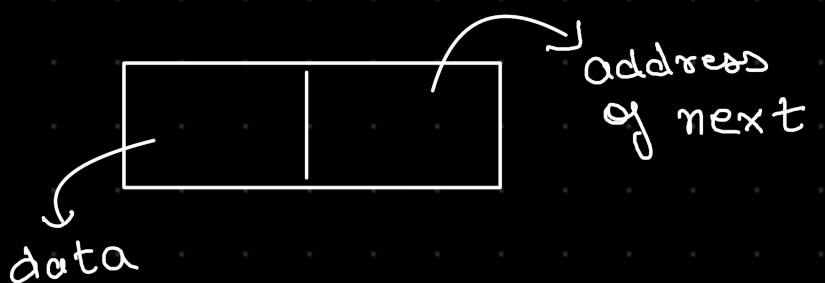
to explain continous / non continous.

how to access?

1. Create a List/Array :-  some problem

2. So we will just store address of 1st and access other elements via that as we will collect them.

   As they are linked ⇒ Linked list.



How to store element?

No inbuilt data structure like this so will make one. data & address of next.

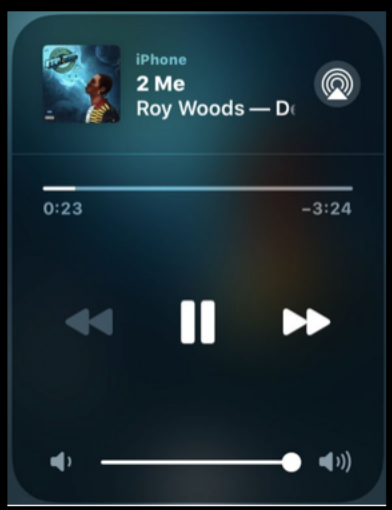
⇒ Classes Node

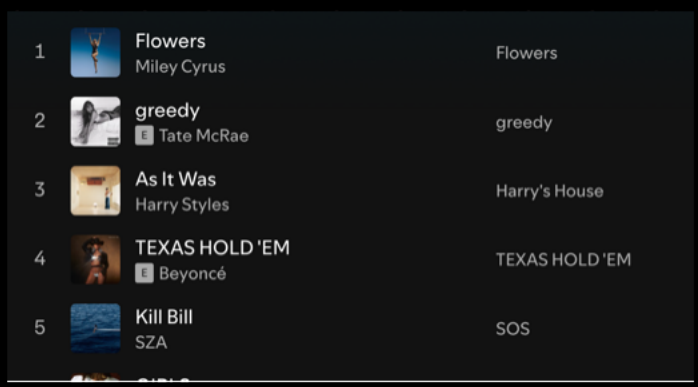
What our node should have.

```
class Node:
    self.data
    self.next
```

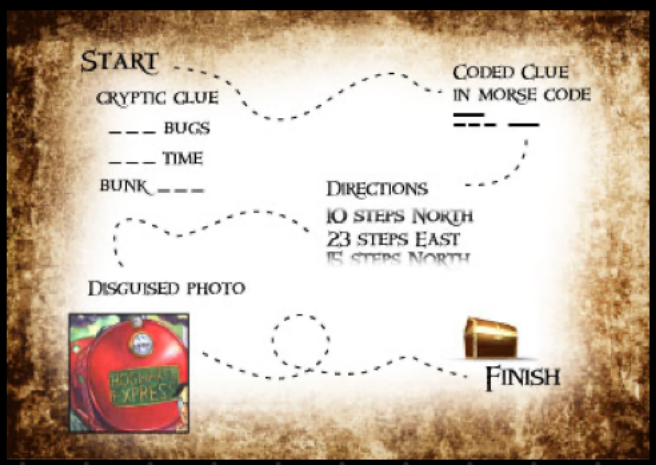
Now before going to implementation, let us look at some examples.

1. Music Playlist





2. Treasure hunt



3. Browsing history

4. Undo / Redo Functionality

Linked List

show using Code1 on how to form manually and join.

explain head & tail as well.

If head == NULL
implies linked list empty.

Print a LL code

first explain using print in file → how first will become null
and then @ via function

Take Input of Linked list

draw lots of diagram with A → B → C → D

1. Problem when we take head.next
2. O(n2) approach of finding tail everytime (refer as last)

   explain intuitively
   using children example

   calculate complexity as well.

3. Optimized approach using tail, explain intuitively.

Length of a Linked List

Linked List operations

1. Insert - head (append), tail (append), in between (insert).
2. Traverse - Print
3. Delete - head, tail (pop), value (remove), index
4. Search - value, index

Insert

1] Head 2] Tail 3] Index

Make sure to draw, draw draw & explain properly.

* Also compare with array as well.

Also cover how we will have to return the value as well if changes happen to head.

how index - 1 to be found out & how that

Show how connection is going to get created or deleted.

Show both the approaches here by referring to old node & diagram.

a = temp->next

Delete

1. Head
2. tail (pop)
3. value (remove)
4. Index

Make sure to explain recursive approach properly + Python tutor & visualization

Special focus on none logic

everything using python tutor

Search in a linked list

1. Value

2. Index

	Array	vs	LL
Access	O(1)		O(n)
Memory management	Fixed size some vacant some fill memory fragmentation not possible		memory fragmentation possible - No unused memory - extra memory for pointers
Insertion	O(n) (start) O(1) (end, not full) O(n) (end, full) middle or ith O(n/2), O(n)		O(1) O(n) O(n)
	good for indexing		insert/delete easy from head
	read fast		write fast
	read heavy		write heavy.

Remove Duplicates

Print reversed linked list

Replace max with value

Mid point of LL

(compare with array)

1. len and then len/2
   (for both odd & even)
   why len - 1 / 2

2. without len ★ intuition

   slow

   fast

   slow.next
   not null ★

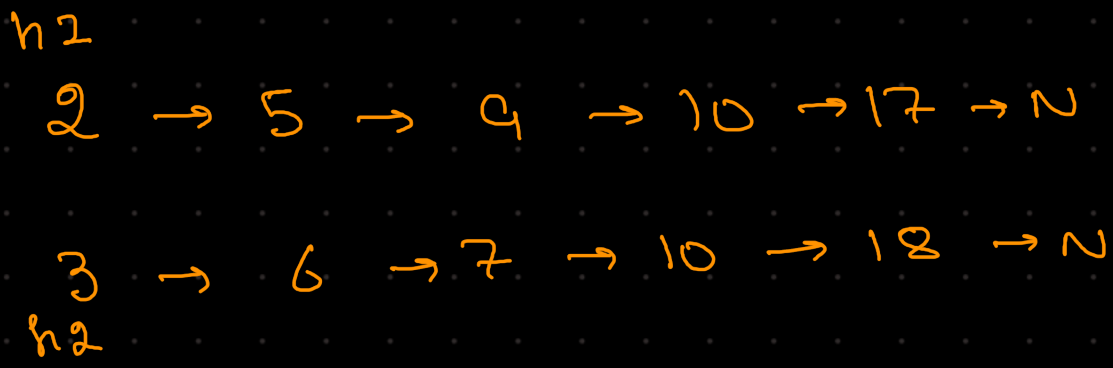
   fast.next.next

   Odd fast → None

   even fast → next → None

draw & explain for both the cases.

Merge 2 sorted link list



jH jt

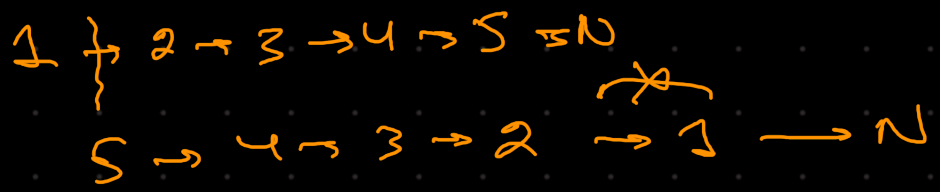
* focus on making sure no None encountered.

merge sort will do in Questions part

Reverse a linked list

1. Using recursion

1. Base case
2. Recursive call → smaller input
3. Small calculation.



explain using Diagram

find complexity

Reverse LL ⇒ return (head & tail)

Write code with visualization and explain properly

how own class as well can be returned

Reverse LL (smart)

4 - 3 - 2 - null



how it is present in our 1 only

Reverse LL

Reverse LL iterative

1 → 2 → 3 → 4 → N

p c n

n = c → next

c → next - p

p = c

c = n

Stop when current now head at p